

csiro-biomass

5th place solution

Rank	5
Team	Kinosuke
Author	Kinosuke
Topic ID	671211
Version	Original

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671211>

Retrieved with Kaggle CLI on 2026-07-18.

5th Place Solution

Thank you to the hosts for this competition. It was a straightforward image task and I really enjoyed participating.

Overview

DINOv3's dense features turned out to be extremely powerful. My approach focused on leveraging these features effectively while preserving the pretrained weights as much as possible. Using Global Features (CLS) would discard DINOv3's powerful local separation capability, so I designed the model to be as close to a segmentation task as possible.

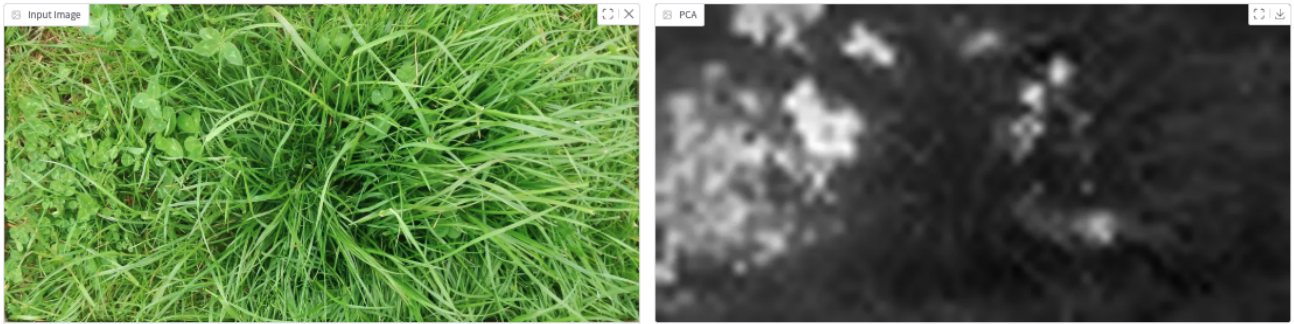
- End-to-end training using DINOv3's dense features
- Gradual Unfreeze up to 50% — no full fine-tuning
- Split 2000x1000 images in half and halve the labels to double the training data
- For shake robustness: ensemble of 6 models x 2 seeds across Large/Huge+ and multiple resolutions
- Inference optimizations (T4x2 parallel inference, FP16) to fit within the 9h time limit

model	resolution	CV (OOF R ²)	Public	Private
DinoV3 Huge+ (child-exp012)	672	0.809	0.75	0.65
DinoV3 Huge+ (child-exp017)	768	0.816	0.75	0.65
DinoV3 Huge+ (child-exp026)	672 + PatchDropout	0.811	0.75	0.65
DinoV3 Huge+ (child-exp032)	864	0.814	0.75	0.65
DinoV3 Large (child-exp020)	864	0.818	0.73	0.66
DinoV3 Large (child-exp037)	960	0.819	0.74	0.65

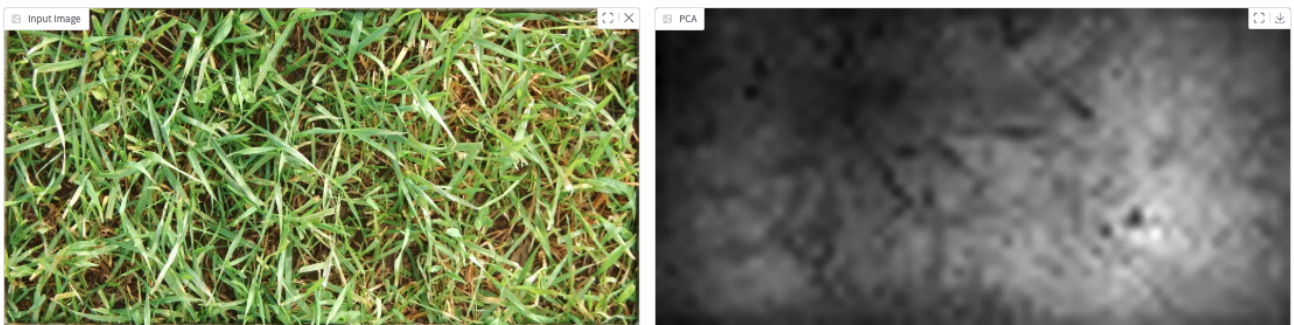
Model

Why Density Estimation?

When visualizing DINOv3 features with PCA, you can see that even zero-shot, DINOv3 excels at local separation. It can clearly distinguish Green, Clover, Dead (partially), and Soil. Focus on Clover:



Focus on Dead:



Note: I intentionally selected samples where the separation is visually clear. Not all samples separate this cleanly.

Despite this excellent local separation ability, attaching a head to the CLS token discards spatial information, which felt wasteful. Instead, I adopted a Density Estimation approach: predict local biomass density from each patch token and sum (integrate) across the entire image.

Architecture: Density-Only Head

```

Input image (1000x1000, one half of left-right split)
  | resize
Input (672x672x3)
  |
DINOv3 ViT-Huge+ backbone (frozen -> gradual unfreeze)
  |
Patch tokens [B, 1764, 1280] (42x42 patches, 1280-dim each)
  | reshape
[B, 1280, 42, 42]
  |
1x1 Conv (1280 -> 5)  <- Only 6,405 learnable parameters here
  |
Softplus (non-negativity constraint: biomass density is physically non-negative)
  |
Density map [B, 5, 42, 42] (local density for 5 targets per patch)
  |
Sum (aggregate all patches)
  |
Prediction [B, 5] = [Green, Dead, Clover, GDM, Total]

```

Key points:

- **CLS token is not used.** Prediction relies solely on patch tokens (Density-Only)
- **1x1 Conv** linearly transforms each patch's 1280-dim features into 5 biomass density values (no spatial convolution)
- **Softplus** enforces non-negativity, with each patch representing "how much biomass is in this region"
- The final **sum over all patches** encodes the physical relationship: biomass = density x area Getting training to converge was challenging. I tried Conv3x3 to learn relationships between neighboring patches, but couldn't resolve NaN loss issues regardless of learning rate adjustments. Training only stabilized with the simpler Conv1x1. I also had to monitor training for the first 5-8 epochs to check if the validation score stabilized, manually stopping runs that didn't converge.

Resolution

Resolution was critical. Higher resolution consistently improved both CV and LB scores. I started at 224 and eventually reached 960. Higher resolution requires more training time and VRAM, so I started experiments at 256 and gradually increased to 512, then 768 in the final 3 weeks of the competition. The final models used Large at 960px and Huge+ at 864px. Training Huge+ at 864px took 20 hours for 5 folds, using 76GB VRAM at batch size 8 (Colab A100 80GB).

Training Strategy

Gradual Unfreeze was key. With a model like DINOv3 that has strong pretrained weights, full fine-tuning from the start destroys the valuable pretrained representations.

Gradual Unfreeze

- **Epochs 1-5:** Train only the head with a higher learning rate. Backbone is frozen.
- **Epochs 6-40:** Gradually unfreeze layers starting from those closest to the head. For example, DINOv3 Large has 24 Transformer blocks in total. With max_unfreeze_ratio=50%, up to 12 blocks are unfrozen by epoch 40. An important design choice was **not using a learning rate scheduler**. Most schedulers apply warmup followed by gradual decay, but with this strategy, layers unfrozen in later epochs wouldn't receive sufficient learning. Instead, I linearly increased the backbone learning rate:
 - Start (epoch 5): 1e-5
 - End (epoch 40): 3e-5

Data Split (Left-Right Halving)

Many public notebooks split the 2000x1000 image into left and right halves, then concatenated the final layer outputs before feeding them to the head. My approach was different: split into left and right halves **and halve the labels too**, effectively doubling the training data. This approach consistently outperformed the 2-stream concatenation method across DINOv3, EVA02-CLIP, and SigLIP.

Loss Function

The base loss was SmoothL1Loss, predicting all 5 targets. The key addition was **ConsistencyLoss**, which enforces the physical relationships:

- Total = Green + Dead + Clover
- GDM = Green + Clover I initially thought predicting only 3 independent targets (Green, Dead, Clover) and deriving the rest would be more efficient. However, since there appeared to be some label noise between Clover and Green, predicting all 5 with a consistency constraint served as effective regularization.

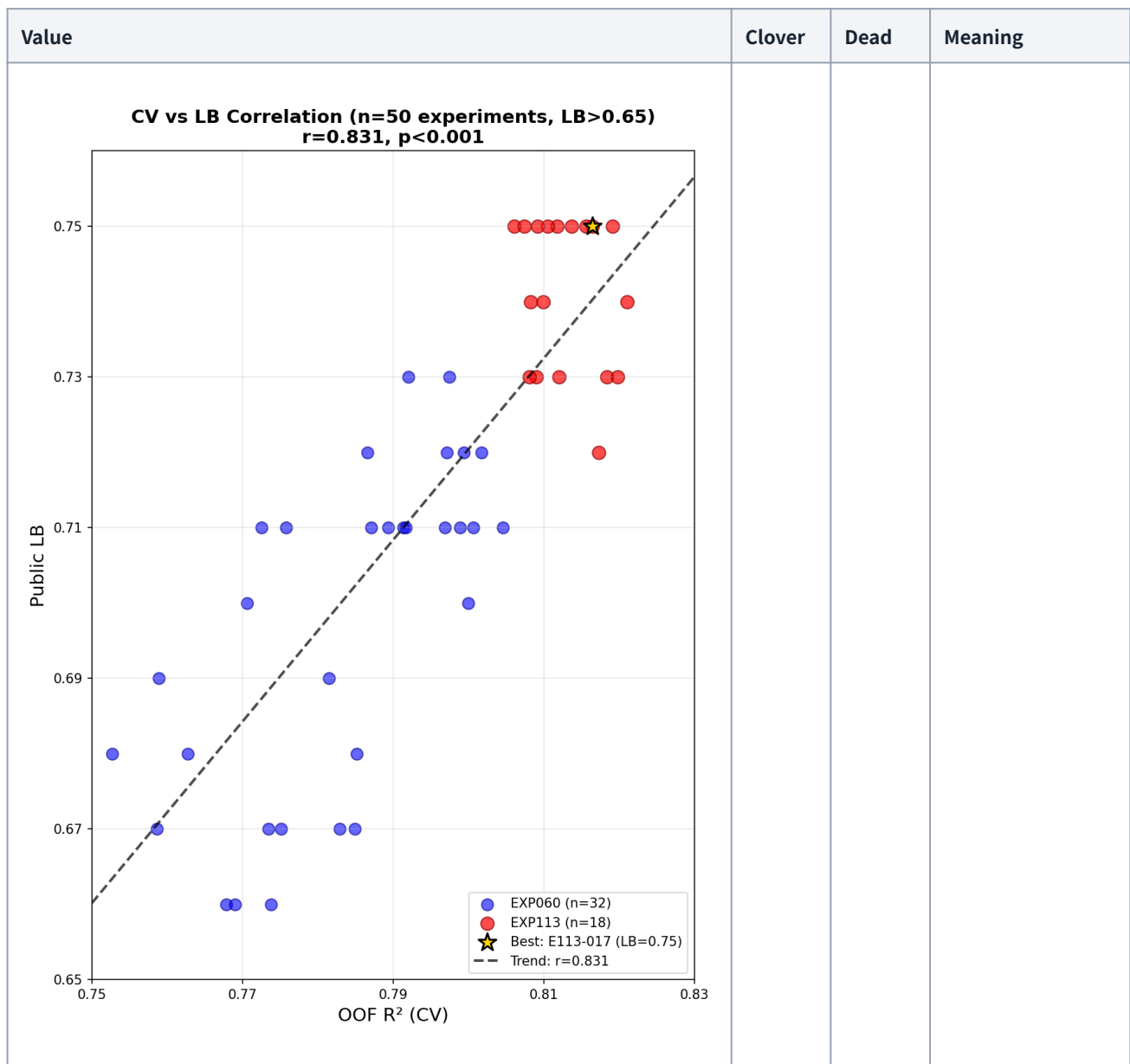
CV Strategy

Adopting the group-based approach from [this discussion post](#) stabilized the CV-LB correlation. Since Clover and Dead were difficult to predict, I used stratification to distribute them evenly across folds:

```
df_wide['clover_dead_presence'] = (  
    (df_wide['Dry_Clover_g'] > 0).astype(int).astype(str) + '_' +  
    (df_wide['Dry_Dead_g'] > 0).astype(int).astype(str)  
)
```

This was used as the stratification variable for StratifiedGroupKFold, ensuring samples with zero Clover or Dead values were evenly distributed across folds. Since many samples have zero values for these targets, naive random splitting could cause certain folds to be skewed.

Value	Clover	Dead	Meaning
0_0	None (=0)	None (=0)	Both Clover and Dead are zero
0_1	None (=0)	Present (>0)	Dead only
1_0	Present (>0)	None (=0)	Clover only
1_1	Present (>0)	Present (>0)	Both present
This made the CV-LB correlation very stable.			



Augmentation

The augmentations used:

```
HorizontalFlip (p=0.5)
VerticalFlip (p=0.5)
RandomRotate90 (p=0.5)
Rotate (limit=10, p=0.3)
RandomResizedCrop (scale=0.85-1.0, ratio=0.95-1.05, p=0.5)
ColorJitter (brightness=0.3, contrast=0.3, saturation=0.3, hue=0.15, p=0.7)
RandomGamma (gamma_limit=80-120, p=0.3)
RandomBrightnessContrast (brightness=0.25, contrast=0.25, p=0.5)
GaussianBlur (blur_limit=3-5, p=0.2)
RandomShadow (p=0.1)
RandomToneCurve (p=0.2)
```

Regarding RandomResizedCrop: I initially thought it shouldn't be used since it changes the amount of grass visible in the image. However, it actually improved accuracy. This may be because the camera-to-ground

distance varies across photographers, and this augmentation improved robustness to such variation. Overly aggressive augmentations degraded performance:

- The host's paper mentioned images were captured with various camera types, so I tried adding diverse noise augmentations, but going too far hurt performance.
- RandomGridShuffle also did not help.

Shake Mitigation

2-Seed Ensemble

Given the small dataset and regression task, a significant shake was expected. Inspired by [Psi's discussion from the Feedback Prize competition](#), I trained and evaluated with 2 seeds to verify that models achieved genuine generalization and to improve robustness.

Model Comparison

I participated from early in the competition, progressively upgrading models: SigLIP -> EVA02-CLIP -> DINOv3. I didn't know DINOv3 would be this strong at the start. I usually prefer CLIP-based models, but DINOv3 was overwhelmingly superior in this competition. All models below used the same training pipeline (augmentation, learning rate, gradual unfreeze). Only batch size was adjusted to fit VRAM.

Model	CV	Public	Private
SigLIP	0.75	0.66	0.55
EVA02-CLIP	0.75	0.69	0.584
DINOv3 [Global Feature]	0.79	0.73	0.64
DINOv3 [Dense Feature]	0.81	0.75	0.66

What Didn't Work

- **TTA** — No improvement
- **Sliding Window Inference** — Instead of splitting left/right, sliding by 500px for 4 passes. Did not help.
- **Data Cleaning** — The "WA/2015-8-21" date had notably noisy labels. Hand-correcting them improved CV but worsened LB. The test data likely contains the same labeling errors.
- **Auxiliary Losses** — Using metadata (Height, Species, Season) as auxiliary targets did not help.
- **Depth Anything** — Depth maps showed strong correlation with Height in train.csv. I tried multiplying depth with the density map to incorporate per-patch height information, but it didn't improve results.
- **EMA** — Did not work, though my implementation may have been incorrect. I didn't investigate further.